Studio Nx

백엔드 233 — 트랙 서비스 (트랙·권한)

2026-10-18

Backend(FastAPI/Python) 233 — `track_service` 트랙과 권한 (목록 · 만들기 · 조회 · 고치기 · 지우기)

항목	내용
파일	`studionx/backend/app/services/track_service.py`
작업	S1-09 기본 악기·이펙트: 트랙 다섯 함수와 권한 도우미 (PRD 02 FR-251, FR-252, FR-269)
작성	Claude (AI), 2026-10-18, 사용자 검토
이전 · 다음	232 플러그인 서비스 · 234 악기 칸

1. 하는 일

트랙 한 줄을 다루는 함수 다섯 개와, 그 아래 모든 작업이 함께 쓰는 권한 도우미 두 개를 둡니다.

함수	하는 일	권한
`_find_track`	트랙과 그 프로젝트의 내 멤버 행을 함께 찾습니다	없거나 남의 것이면 404
`_require_editor_track`	쓰기용. 권한이 없으면 403	OWNER, EDITOR
`list_tracks`	프로젝트의 트랙 목록 (`order_index`, `track_id` 순)	멤버면 누구나
`create_track`	트랙 만들기. 부모가 있으면 같은 프로젝트의 트랙인지 확인	OWNER, EDITOR
`require_track`	트랙 한 건 (조회용)	멤버면 누구나
`update_track`	보낸 칸만 바꿉니다	OWNER, EDITOR
`delete_track`	트랙 지우기. 악기·이펙트·연결선은 DB CASCADE 로 함께 지워집니다	OWNER, EDITOR

권한 규칙은 `project_service`(P1-13)와 같습니다. 조회는 멤버면 누구나이되 **남의 프로젝트는 있는지조차 404 로 숨기고**, 쓰기는 OWNER·EDITOR 만 403 입니다.

2. 언제 쓰나

쓰는 곳	
`GET/POST /projects/{id}/tracks`, `GET/PATCH/DELETE /tracks/{id}` (238)	다섯 라우터가 이 함수들을 부릅니다
`list_instruments` · `list_effects` · `list_links` (234~236)	먼저 `require_track` 으로 조회 권한을 확인합니다
`_require_instrument` · `_require_effect` (234·235)	`_require_editor_track` 으로 쓰기 권한을 확인합니다
프로젝트 만들기 API (P1-13)	MASTER 트랙을 만들 때 `create_track` 을 씁니다 (FR-252)

3. 코드

권한 도우미

```
def _find_track(db: Session, user: User, track_id: int):
    """트랙과 그 프로젝트·내 멤버 행을 함께 찾는다. 없거나 남의 것이면 404."""
    track = db.get(Track, track_id)
    if track is None:
        raise AppError(ErrorCode.NOT_FOUND, 404, "Track not found.")
    project, member = project_service.get_membership(db, track.project_id, user)
    return track, project, member


def _require_editor_track(db: Session, user: User, track_id: int) -> Track:
    """쓰기가 필요한 작업용: 남의 트랙은 404, 권한이 없으면 403."""
    track, _project, member = _find_track(db, user, track_id)
    project_service.require_role(member, MemberRole.OWNER, MemberRole.EDITOR)
    return track
```

목록과 만들기

```
def list_tracks(db: Session, user: User, project_id: int) -> Sequence[Track]:
    project_service.get_membership(db, project_id, user)
    stmt = (
        select(Track)
        .where(Track.project_id == project_id)
        .order_by(Track.order_index, Track.track_id)
    )
    return db.execute(stmt).scalars().all()


def create_track(db, user, project_id, *, name, track_type=TrackType.MIDI, color="#7C5CFF",
                 volume_db=Decimal("0.00"), pan=Decimal("0.00"), is_muted=False, is_solo=False,
                 is_enabled=True, routing_mode=RoutingMode.CHAIN, order_index=0,
                 parent_track_id=None) -> Track:
    project, member = project_service.get_membership(db, project_id, user)
    project_service.require_role(member, MemberRole.OWNER, MemberRole.EDITOR)
    if parent_track_id:
        parent = db.get(Track, parent_track_id)
        if parent is None or parent.project_id != project.project_id:
            raise AppError(ErrorCode.BAD_REQUEST, 400, "Invalid parent track.")
    track = Track(project_id=project.project_id, name=name, track_type=track_type, ...)
    db.add(track)
    db.commit()
    db.refresh(track)
    return track
```

고치기와 지우기

```
def update_track(db: Session, user: User, track_id: int, **kwargs) -> Track:
    track = _require_editor_track(db, user, track_id)
    for key, value in kwargs.items():
        if value is not None and hasattr(track, key):
            setattr(track, key, value)
    db.commit()
    db.refresh(track)
    return track
```

```
def delete_track(db: Session, user: User, track_id: int) -> None:
    track = _require_editor_track(db, user, track_id)
    db.delete(track)
    db.commit()
```

문법 풀이

식	뜻
`db.get(Track, track_id)`	기본 키로 한 건을 찾습니다. 없으면 None
`project_service.get_membership(db, project_id, user)`	프로젝트와 내 멤버 행을 돌려줍니다. 남의 것이면 여기서 404 가 납니다
`require_role(member, MemberRole.OWNER, MemberRole.EDITOR)`	둘 중 하나라도 아니면 403 입니다. 역할이 여러 개일 때 이렇게 나열합니다
`_project`, `_member`	안 쓰는 값이라는 표시입니다. `_find_track` 이 셋을 돌려주므로 자리만 받습니다
`**kwargs` 로 받고 `hasattr` 로 거름	보낸 칸만 바꾸는 방식입니다. 모델에 없는 이름이 와도 무시합니다
`if value is not None`	None 을 보내도 그대로 두는 규칙입니다. 라우터가 안 보낸 칸도 None 이라 함께 걸러집니다
`db.commit()` 뒤 `db.refresh()`	실제 DB 에 남기고, DB 가 채운 값(기본값·`updated_at`)까지 다시 읽어 응답에 담습니다

4. 왜 이렇게

선택	이유
인자를 `(db, user, ...)` 로 통일	P1-13 프로젝트 서비스와 같은 모양입니다. 서비스마다 순서가 다르면 실수가 납니다
남의 트랙을 404 로	403 을 돌려주면 "그 번호의 트랙이 있다" 는 사실이 새어 나갑니다. 프로젝트 서비스와 같은 규칙입니다
쓰기 뒤 `commit`	`get_db` 는 commit 을 하지 않습니다. 서비스가 직접 커밋하지 않으면 응답은 나가지만 DB 에 남지 않습니다
목록 정렬에 `track_id` 를 덧붙임	순서 번호가 같을 때(새로 만든 직후) 화면 순서가 흔들리지 않게 합니다
부모 트랙을 같은 프로젝트로 제한	다른 프로젝트의 트랙을 부모로 넣으면 그룹 접기/펼치기가 꼬입니다 (FR-252)
삭제를 DB 에 맡김	악기·이펙트·연결선 세 표가 `ON DELETE CASCADE` 입니다. 앱이 빠뜨릴 일이 없습니다

5. 시험

`tests/test_track_routes.py` 가 다음을 확인합니다.

- 목록 순서(`order_index`, `track_id`), 만들기 201과 기본값
- 색 형식 오류와 볼륨 범위 초과는 422
- 고치기 응답(이름·볼륨·`routing_mode` = `PATCHER`), 지운 뒤 404
- 남의 프로젝트 트랙은 조회·목록·만들기 모두 404
- VIEWER 는 조회 200, 쓰기(만들기·고치기·지우기) 403